Aprendizaje automático avanzado
Transformadores

Juan Miguel Santos

Laboratorio de investigación y desarrollo experimental en computación (LIDEC)
Universidad Nacional de Hurlingham

(1 de octubre de 2026)

Transformador

Bloque de transformación

- De modo similar que expresamos las transformaciones adentro del bloque mediante variables auxiliares $T \in R^{N \times d}$, podemos expresar las transformaciones dentro del bloque de cada *token* i usando variables intermedias $t \in R^{1 \times d}$:

$$
\begin{aligned}
t_i^1 &= autoAtencionMultiple(x_i, [x_1, x_2, ..., x_N]) \\
t_i^2 &= x_i + t_i^1 \\
t_i^3 &= Normalizacion(t_i^2) \\
t_i^4 &= PMC(t_i^3) \\
t_i^5 &= t_i^3 + t_i^4 \\
h_i &= Normalizacion(t_i^5).
\end{aligned}
$$

Transformador

Bloque de transformación.

- Supongamos que el token j influye mucho en la nueva representación del token i (esto se debe a que $q_i k_j$ es muy alto en relación a otros $q_i k_{j'}$, $j \neq j', j \neq i$).
- Luego, la salida del bloque de transformación introducirá información de token j en la nueva representación del token i.

Transformador

Bloque de transformación.

- En un nuevo bloque de transformación, consideremos que la nueva representación de la entrada i pesa en la de otro token m (ya que el q de la nueva representación de i tiene mucha similitud con el k de la representación del token m).

Transformador

Bloque de transformación.

- Entonces esta nueva representación del token i (que traía información del token j de la capa de transformación anterior) va a pasar a la representación de salida del token m en el segundo bloque de transformación.
- Y así a través de las sucesivas bloques.

Transformador

Codificación de la posición.

- La posición de las representaciones contextuales en una entrada aportan información acerca de la relación entre ellas.
- A tal efecto, se le suma a cada representación contextual (*token*) un vector de la misma dimensión (d) cuyo valor depende de la posición del *token* en la secuencia de entrada.

Transformador

Codificación de la posición.

- La función para obtener dichos vectores que dependen de la posición es

$$vp(p,j) = \begin{cases} sen\left(\frac{p}{10000^{\frac{2j}{d}}}\right) & \text{si} \quad \text{j es par} \\ \\ cos\left(\frac{p}{10000^{\frac{2j-1}{d}}}\right) & \text{si} \quad \text{j es impar} \end{cases}$$

donde d es la dimensión de la representación contextual, p la posición en la secuencia de entrada del transformador, j el elemento dentro del vector posicional y 10000 es una constante considerado un hiper parámetro que es determinado experimentalmente.

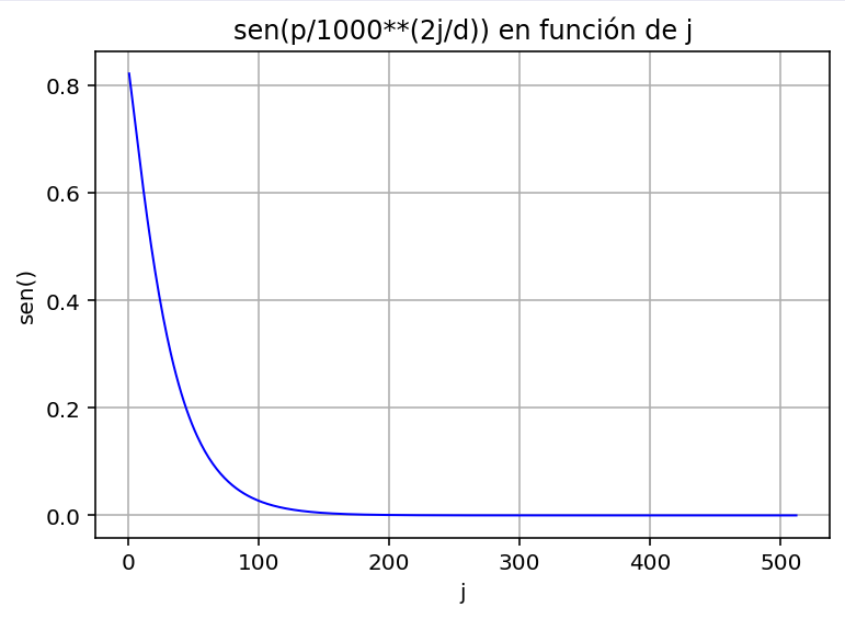
Transformador

Codificación de la posición.

- Tomemos el valor de p en 1.
- Fijarse que j varía entre 1 y d, por lo tanto el exponente de $10000^{\frac{2j}{d}}$ crece a medida que j crece.
- Entonces, a medida que j crece, el cociente $\frac{p}{10000^{\frac{2j}{d}}}$ se hace más pequeño.
- De esto resulta que los elementos pares del vp tendrán un valor más pequeño a medida que j se acerca a d.

Transformador

Codificación de la posición.



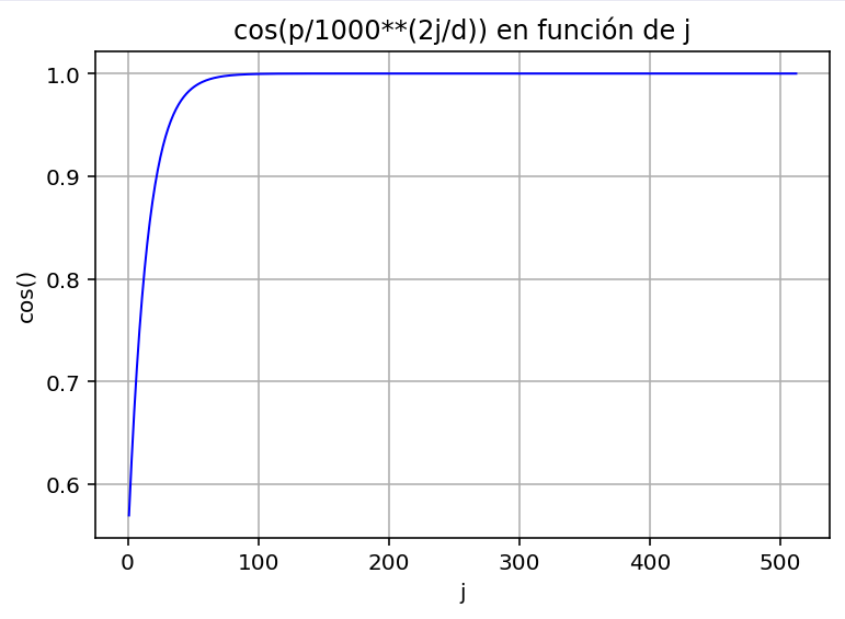
Transformador

Codificación de la posición.

- Lo inverso ocurre con los elementos impares de vp dado que a medida que el cociente $\frac{p}{10000^{\frac{2j-1}{d}}}$ se acerca a 0, el valor de la función $cos()$ aumenta.
- También es útil notar que el valor de 10000 garantiza que tanto los argumentos de $sen()$ y $cos()$ se mantengan

Transformador

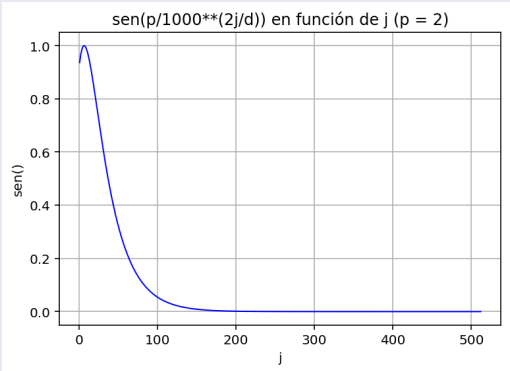
Codificación de la posición.

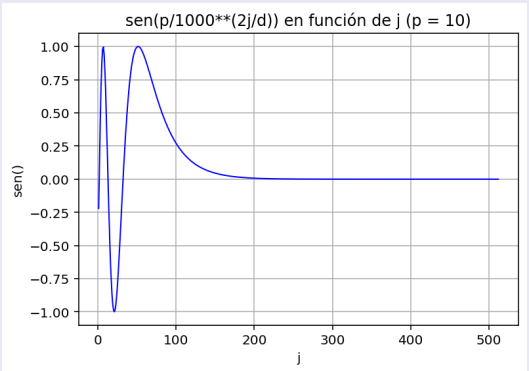


Transformador

Codificación de la posición.

- Pero ¿qué ocurre si $p \neq 1$? Tomemos el valor de p en 2 o en 10.

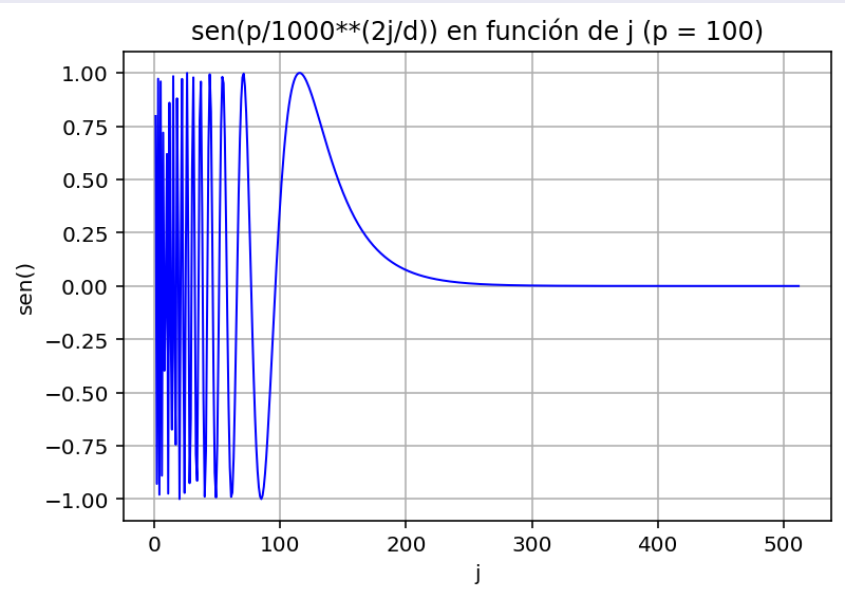




Transformador

Codificación de la posición.

- ¿Y si p es 100?



Transformador

Codificación de la posición.

- Lo que podemos observar es que, a medida que p aumenta, el valor de las componentes pares del vector posicional se hacen 0 (o tienden a 0) más tarde.
- Algo similar ocurre con las componentes impares que se hacen 1 (o tienden a 1) más tarde.
- Esto es, la representación de los vectores posicionales la podemos ver como vectores con valores de las funciones $sen()$ y $cos()$ seguidos de valores muy cercanos a 0 (j pares) y 1 (j impares).

Transformador

Codificación de la posición.

Por ejemplo, para $d = 32$ y $p = 1$ tenemos:
0.31 0.97 0.10 1 0.03 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0

para $d = 32$ y $p = 8$:
0.57 -0.32 0.72 0.83 0.25 0.98 0.08 1 0.03 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0

y para $d = 32$ y $p = 16$:
-0.94 -0.79 1 0.36 0.48 0.93 0.16 1 0.05 1 0.02 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0

Transformador

Codificación de la posición.

- Tener en cuenta que tanto p como j pertenecen a los naturales (excluye el 0).
- Los tokens que forman parte de la secuencia de entrada al transformador se obtienen de la siguiente manera

$$x_i = rc_i + vp_i$$

donde
x_i es el token de la entrada ubicado en la posición i,
rc_i es la representación contextual del token de la entrada ubicado en la posición i, y
vp_i es el vector posicional del token de la entrada en la posición i.

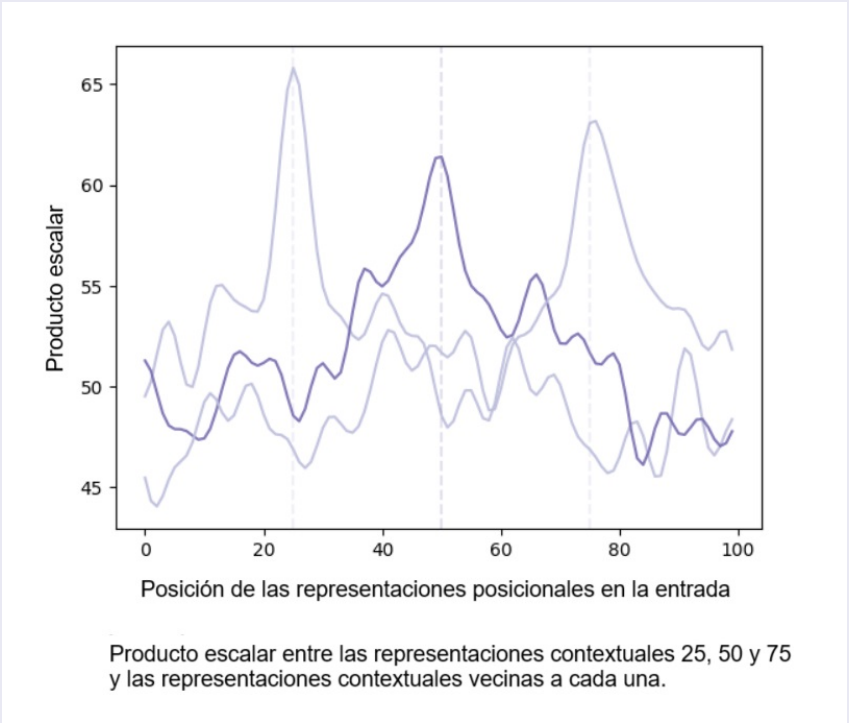
Transformador

Codificación de la posición.

- Una característica de estos vectores posicionales o representaciones posicionales es que mantienen el producto escalar con sus vecinos (representaciones posicionales de posiciones cercanas) con una tendencia decreciente a medida que dichos vecinos se alejan posicionalmente.

Transformador

Codificación de la posición.



Producto escalar entre las representaciones contextuales 25, 50 y 75 y las representaciones contextuales vecinas a cada una.

(Extraída de https://www.kaggle.com/code/hakim11/transformer-encoder-architecture)

Transformador

Normalización. Nota.

- Claramente, el estado de activación de las unidades (neuronas artificiales) en las capas de una RNA dependen de las entradas y los pesos en la capa.
- Durante el entrenamiento, los pesos en las capas varían y pueden producir que los valores de activación tomen valores extremos.

Transformador

Normalización. Nota.

- El algoritmo de retropropagación no funciona eficientemente (es más lento) cuando las funciones de activación toman valores 'muy' cercanos al valor que asintóticamente se acercan.

Transformador

Normalización. Nota.

- O directamente, la derivada en una región plana (por ejemplo, para valores negativos de entrada en la ReLU) se hace nula.
- También, en el caso que la función sea lineal, los valores de los pesos pueden crecer de una manera divergente.

Transformador

Normalización. Nota.

- De modo que, se ha comprobado experimentalmente que mantener los valores de activación de las unidades acotados acelera el aprendizaje.

Transformador

Normalización. Nota.

- Además, la distribución de las entradas de una capa puede cambiarse en la salida, típicamente por desplazamiento o escala.
- Esto último es conocido como Desplazamiento covariado interno (o *Internal Covariate Shift* en inglés).

Transformador

Normalización. Nota.

- Para evitar estos problemas se puede usar
  - **Normalización por lotes** (o *Batch normalization* en inglés), o
  - **Normalización por capa** (*Layer normalization* en inglés).

Transformador

Normalización por lotes. Nota.

- En la normalización por lotes, cada columna de un lote (si, por ejemplo, el tamaño del lote es 10, cada columna tiene 10 elementos) es estandarizada dividiendo por la media μ de los valores de la columna y dividida por la desviación estándar σ.

Transformador

Normalización por lotes. Nota.

- Sea X un lote con ejemplos, $x_1, x_2, ..., x_L$, donde L es el tamaño del lote, y cada ejemplo x_i es
$$x_i = (x_{i,1}, x_{i,2}, ..., x_{i,m})$$
donde m es la dimensión de cada ejemplo, entonces la media y desviación estándar de la columna j es
$$\mu_j = \frac{1}{L} \sum_{i=1}^{N} x_{i,j}$$
$$\sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^{L} (x_{i,j} - \mu_j)^2}$$
.

Transformador

Normalización por lotes. Nota.

- Así, en vez de usar el valor de $x_{i,j}$ se usará el valor de $x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j}$.
- Esto vale para cualquier lote, ya sea un lote entrada o un lote obtenido a la salida de una capa de la red.

Transformador

Normalización por lotes. Nota.

- Además, para evitar el escalamiento y desplazamiento, el valor obtenido se transforma según:
$$x''_{i,j} = x'_{i,j}\gamma_j + \beta_j$$
donde γ_j tiene un valor inicial de 1 y β_j de 0.
- Tanto γ_j como β_j son parámetros que se ajustan durante el aprendizaje (al igual que los pesos de las conexiones) siguiendo el método del gradiente descendente (derivar respecto a γ_j y β_j).

Transformador

Normalización por capa. Nota.

- Consideremos una capa con unidades cuya función de activación es $g()$ y nombraremos $V_1, V_2, ..., V_m$ a los estados de activación de dichas unidades.
- Consideremos ahora que a dicha capa le sigue una capa de normalización cuyas entradas son $V_1, ...V_m$ y salidas las notamos con $V_1'', V_2'', ..., V_m''$.

Transformador

Normalización por capa. Nota.

- Cada salida V_i'' será calculada como
$$V_i'' = V_i' \gamma_i + \beta_i$$
donde V_i' está definida como
$$V_i' = \frac{V_i - \mu}{\sigma}$$

con $\mu = \frac{1}{m} \sum_{j=1}^{m} V_j$ y
$$\sigma = \sqrt{\frac{1}{m} \sum_{j=1}^{m} (V_j - \mu)^2}.$$

Transformador

Normalización por capa. Nota.

- Para actualizar el valor de cada γ_i usaremos el gradiente descendente

$$\gamma_i^{nuevo} = \gamma_i^{anterior} - \eta \frac{\partial E}{\partial \gamma_i}$$

donde $\frac{\partial E}{\partial \gamma_i} = \frac{\partial E}{\partial V_i''} \frac{\partial V_i''}{\partial \gamma_i} = \frac{\partial E}{\partial V_i''} V_i'$.

- Para actualizar el valor de cada β_i procedemos en forma similar

$$\beta_i^{nuevo} = \beta_i^{anterior} - \eta \frac{\partial E}{\partial \beta_i}$$

donde $\frac{\partial E}{\partial \beta_i} = \frac{\partial E}{\partial V_i''} \frac{\partial V_i''}{\partial \beta_i} = \frac{\partial E}{\partial V_i''}$.

Transformador

Normalización por capa. Nota.

- Sin embargo, los efectos de γ_i, β_i, μ y σ llegan al cálculo de la actualización de los pesos de las conexiones que llegan a la capa cuyas salidas son $V_1, V_2, ..., V_m$.
- Sea $w_{i,j}$ el peso de una conexión que llega a la unidad i de la capa en cuestión.
- Escribimos la actualización del peso de la conexión $w_{i,j}$ como

$$w_{i,j}^{nuevo} = w_{i,j}^{anterior} - \eta \frac{\partial E}{\partial w_{i,j}}.$$

Transformador

Normalización por capa. Nota.

- Para calcular $\frac{\partial E}{\partial w_{i,j}}$ es útil calcular $\frac{\partial E}{\partial h_i}$:

$$\frac{\partial E}{\partial h_i} = \frac{\partial E}{\partial V_i''} \frac{\partial V_i''}{\partial V_i'} \frac{\partial V_i'}{\partial V_i} \frac{\partial V_i}{\partial h_i}.$$

- La $\frac{\partial V_i''}{\partial V_i'}$ es directamente γ_i.
- Sin embargo, $\frac{\partial V_i'}{\partial V_i}$ requiere un poco más de cuidado ya que V_i' depende de V_i pero también de μ y de σ y estas dos últimas a su vez de V_j donde $1 \leq j \leq m$ donde m es el número de unidades de la capa.

Transformador

Normalización por capa. Nota.

- La $\frac{\partial V_i'}{\partial V_i}$ es

$$\frac{\partial V_i'}{\partial V_i} = \frac{\partial \frac{V_i - \frac{1}{m}\sum_{j=1}^{m} V_j}{\sqrt{\frac{1}{m}\sum_{j=1}^{m}(V_j-\mu)^2}}}{\partial V_i}.$$

- Como es la derivada de un cociente calculamos primero $\frac{\partial(V_i - \frac{1}{m}\sum_{j=1}^{m} V_j)}{\partial V_i}$ que da $(1 - \frac{1}{m})$.
- Luego calculamos $\frac{\partial \sqrt{\frac{1}{m}\sum_{j=1}^{m}(V_j-\mu)^2}}{\partial V_i}$ que da

$$\frac{1}{\sigma}\frac{1}{m}((-2)\frac{m-1}{m} + 2(1 - \frac{1}{m})) = \frac{1}{\sigma}\frac{1}{m}(-2 + \frac{2}{m} + 2 - \frac{2}{m}) = 0.$$

Transformador

Normalización por capa. Nota.

- Finalmente, la $\frac{\partial V_i'}{\partial V_i}$ es
$\frac{\partial V_i'}{\partial V_i} = \frac{(1-\frac{1}{m})\sigma}{\sigma^2} = \frac{1-\frac{1}{m}}{\sigma}$.
- Como la $\frac{\partial h_i}{\partial w_{i,j}}$ es V_j podemos completar el cálculo de actualización de $w_{i,j}$ ya que
$$\frac{\partial E}{\partial w_{i,j}} = \frac{\partial E}{\partial h_i}\frac{\partial h_i}{\partial w_{i,j}}.$$

Transformador

Uso como herramienta. Matriz E.

- La matriz de representaciones contextuales se nota E la cual tiene una representación contextual por cada fila.
- Así, $E \in R^{|V| \times d}$.
- Cuando una entrada es presentada al Transformador, se deben obtener N representaciones contextuales correspondientes a las N palabras de la entrada.

Transformador

Uso como herramienta. Matriz E.

- Para obtener una representación contextual en la entrada (*tokens*), para cada palabra se identifica el índice en el vocabulario y luego se obtiene la representación contextual tomando la fila de la matriz E indicada por el índice.

Transformador

Uso como herramienta. Matriz E.

- Una alternativa es usar representaciones de las palabras mediante la codificación *one-hot* $\in R^{1\times|V|}$.
- Luego, multiplicamos la representación *one-hot* por la matriz E y obtendremos la representación contextual de la palabra.

Transformador

Uso como herramienta. Matriz E.

- Si queremos obtener las representaciones contextuales de todas las N palabras que están en la entrada, podemos multiplicar la matriz de las N representaciones *one-hot* por la matriz E y obtendremos una matriz de $N \times d$.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Cuando se quiere usar un Transformador para construir un modelo de lenguajes se le debe agregar un cabezal.
- Esto permitirá poder predecir la próxima palabra x_{N+1} dada una cadena de entrada expresada mediante $(x_1, x_2, ..., x_N)$.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Supongamos que tenemos una cadena de entrada
  'Esa persona quiere ocultar su'
  y deseamos predecir la próxima palabra.
- Tenemos que ver cual es la probabilidad para cada palabra del diccionario dada la entrada, esto es:
  $P(p_{prox}/$'Esa persona quiere ocultar su'$)$.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- La forma de hacer esto con el Transformador es obtener la próxima palabra a partir de la última (la N) representación contextual h_N^L de la salida del último bloque de transformación (el L).
- ¿Cómo?

Transformador

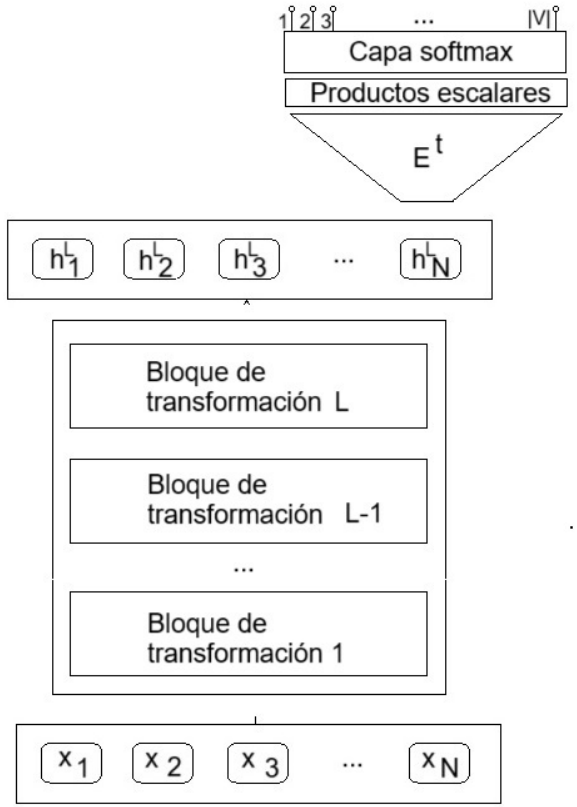
Uso como herramienta. Cabezal de modelado del lenguaje.

Nota: Recordar que la representación en la salida del último bloque de codificación para cada *token* en la entrada toma en cuenta todos los *tokens* de la entrada (sucesivas capas de auto atención en la pila de bloques de codificación).

¿Porqué la última representación en la última capa de codificación? Porque el mecanismo en la codificación es **causal** y la última representación es la única que tiene en cuenta todos las representaciones de la entrada.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.



Transformador como predictor de próxima palabra.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Se calcula el producto escalar entre h_N^L y la representación contextual de todas las palabras del diccionario y dicho producto es la entrada de una capa softmax.
- Esto es equivalente a multiplicar $h_N^L \in R^{1\times d}$ por la matriz $E^t \in R^{d\times|V|}$ con lo que se obtiene un vector $\in R^{1\times|V|}$ que es la entrada a la capa softmax.
- Finalmente, podemos leer la salida de la capa softmax como la distribución de probabilidad para $P(p_i/\text{'Esa persona quiere ocultar su'})$

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Si observamos las salidas de la capa softmax encontraremos que la salida correspondiente a la palabra 'papada' es la mayor y completamos la frase:
  'Esa persona quiere ocultar su papada'.
- No siempre puede ser una buena idea elegir la palabra con la mayor probabilidad. Existen otras alternativas.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- La que acabamos de ver se denomina decodificación ansiosa (o *greedy decoding* en inglés).
- Está última podría conducir a la generación de texto demasiado previsible, muy parecido a como lo ha aprendido el Transformador.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Otra alternativa es **tomar una palabra al azar dentro de las k más probables** (o *Top-k sampling* en inglés).
- De la distribución obtenida por la capa softmax, se toman las k más probables y se vuelve a obtener una nueva distribución pero basado en solo las k palabras. Finalmente, se elige una de las k palabras según una probabilidad.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Si las probabilidades obtenidas por la capa softmax no se acumulan en un tope de k palabras (es decir, la distribución es más o menos uniforme a lo largo del diccionario), tomar las k primeras puede llegar a no ser buena idea.

Transformador

Uso como herramienta. Cabezal de modelado del lenguaje.

- Como alternativa se usa una técnica llamada **muestreo sobre un porcentaje de la masa de probabilidades** (*top-p sampling o nucleus sampling* en inglés).
- En este caso, se ordenan las probabilidades obtenidas y se incluyen todas las palabras hasta acumular un porcentaje p con sus probabilidades.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- Dado una secuencia de entrada se busca que la capa softmax del cabezal tenga el mayor valor sobre la unidad que tiene un 1 en la representación *one-hot* de la próxima palabra, es decir, a la palabra que le sigue a la secuencia de entrada.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- Para entrenar un Transformador se usa la entropía cruzada sobre la salida de la capa softmax como función de pérdida o de error:

$$L_{EC} = -\sum_{p \in V} y_p \log \hat{y}_p$$

donde

y_p es la probabilidad esperada en la salida p, y
$\hat{y}_p$ es la probabilidad obtenida para la salida p.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- La distribución esperada es un vector sobre todas las posibles palabras en V.
- Como la distribución esperada se representa de modo *one-hot*, la L_{EC} queda:

$$L_{EC}(\hat{y}, y) = -\log \hat{y}_{p'}.$$

donde p' es la posición dentro de y tal que $y_{p'} = 1$.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- Es decir, la función de pérdida dependerá de cuánto se acerque a 1 la $\log \hat{y}_{p'}$.
- Si $\hat{y}_{p'}$ está muy cerca de 0 (algo que no queremos), $\log \hat{y}_{p'}$ se acercará a $-\infty$ y por el símbolo negativo, la L_{EC} se acercará a ∞.
- En cambio, si $\hat{y}_{p'}$ está muy cerca de 1 (algo que sí queremos), $\log \hat{y}_{p'}$ se acercará a 0 y la L_{EC} se acercará a 0 también.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- Sea un corpus formado por un texto continuo (incluyendo los signos de puntuación) al cual le corresponden N_{corpus} *tokens* (podemos pensarlo como un *token* por palabra o signo de puntuación).
- Consideramos los N primeros *tokens* y los presentamos como entrada al Transformador.
- El Transformador emite una salida softmax de N representaciones contextuales.

Transformador

Modelado de un lenguaje. Auto-supervisión.

- Sobre dicha salida y la representación *one-hot* de la palabra que debería ser predicha, calculamos L_{EC}.
- Se actualizan los parámetros del Transformador.
- La próxima entrada al Transformador estará formada por N *tokens* correspondientes a una ventana de dimensión N corrida 1 palabra a la derecha, y así sucesivamente.